

Web API Hands-On – Answers

Task 1: Web API using Custom Model Class

Step 1: Create Model Classes

Create a Models folder and add the following classes. Employee has a Department and a list of Skills.

Models/Department.cs

```
namespace WebApiDemo.Models { public class Department { public int
DeptId { get; set; } public string DeptName { get; set; } }
}
```

Models/Skill.cs

```
namespace WebApiDemo.Models { public class Skill { public int SkillId
{ get; set; } public string SkillName { get; set; } }
}
```

Models/Employee.cs

```
using System; using
System.Collections.Generic;

namespace WebApiDemo.Models { public
class Employee { public int Id { get;
set; } public string Name { get; set; }
public int Salary { get; set; } public
bool Permanent { get; set; } public
Department Department { get; set; }
public List Skills { get; set; } public
DateTime DateOfBirth { get; set; } }
}
```

Step 2: Create EmployeeController

Create a private helper method GetStandardEmployeeList() that returns a hardcoded list of Employee objects. The GET action method calls this helper. ProducesResponseType(200) is added for Swagger documentation.

Controllers/EmployeeController.cs

```

using System; using
System.Collections.Generic; using
Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc; using
WebApiDemo.Models;

namespace WebApiDemo.Controllers {
[Route("api/[controller]")] [ApiController]
public class EmployeeController :
ControllerBase
{
// Private helper private List GetStandardEmployeeList() {
return new List { new Employee { Id = 1, Name = "Alice",
Salary = 75000,
Permanent = true,
DateOfBirth = new DateTime(1990, 5, 10), Department =
new Department { DeptId=1, DeptName="IT" }, Skills =
new List { new Skill { SkillId=1, SkillName="C#" }, new
Skill { SkillId=2, SkillName="ASP.NET" } } }, new
Employee { Id = 2, Name = "Bob", Salary = 60000,
Permanent = false,
DateOfBirth = new DateTime(1995, 3, 22), Department =
new Department { DeptId=2, DeptName="HR" }, Skills =
new List { new Skill { SkillId=3, SkillName="Excel" } }
}
};
}

// GET api/employee
[AllowAnonymous]
[HttpGet]
[ProducesResponseType(typeof(List),
200)] public ActionResult> GetStandard()
{ var list = GetStandardEmployeeList();
return Ok(list);
}

```

```
// ■ POST api/employee
[HttpPost] public ActionResult Post([FromBody]
Employee emp) { // In real app: save to DB
emp.Id = 100; // mock assigned id return
CreatedAtAction(nameof(GetStandard), emp); }

// PUT api/employee/1
[HttpPut("{id}")] public IActionResult Put(int id,
[FromBody] Employee emp) { // In real app: update record
return NoContent(); }
}
}
```

Step 3: Swagger Output for GET Method

Run the application and open <https://localhost:{port}/swagger>. The GET /api/employee method shows response code 200 with the Employee schema.

[Swagger UI – GET /api/employee Response]

```
GET /api/employee

Response 200 (application/json)

[
{
  "id": 1,
  "name": "Alice",
  "salary": 75000,
  "permanent": true, "department": { "deptId": 1,
  "deptName": "IT" },
  "skills": [
    { "skillId": 1, "skillName": "C#" }, {
    "skillId": 2, "skillName": "ASP.NET" }
  ], "dateOfBirth": "1990-05-
10T00:00:00"
},
{
  "id": 2,
  "name": "Bob",
  "salary": 60000,
  "permanent": false,
  "department": { "deptId": 2, "deptName": "HR" },
  "skills": [ { "skillId": 3, "skillName": "Excel" } ],
  "dateOfBirth": "1995-03-22T00:00:00"
}
]
```

Create a Filters folder and add CustomAuthFilter class that inherits ActionFilterAttribute. Override OnActionExecuting to check the Authorization header.

Filters/CustomAuthFilter.cs

```
using Microsoft.AspNetCore.Mvc;
using
Microsoft.AspNetCore.Mvc.Filters;

namespace WebApiDemo.Filters { public class CustomAuthFilter :
ActionFilterAttribute { public override void
OnActionExecuting(ActionExecutingContext context) { // Check if
Authorization header exists bool hasAuth =
context.HttpContext.Request.Headers.ContainsKey("Authorization");

if (!hasAuth) { context.Result = new
BadRequestObjectResult( "Invalid request
- No Auth token"); return; }

// Check if value contains 'Bearer' string
authValue = context.HttpContext.Request
.Headers["Authorization"];

if (!authValue.StartsWith("Bearer")) { context.Result =
new BadRequestObjectResult( "Invalid request - Token
present but Bearer unavailable"); return; }

base.OnActionExecuting(context
); }
}
}
```

Apply CustomAuthFilter to EmployeeController

Add [CustomAuthFilter] attribute on top of the EmployeeController class. Every request to this controller will now go through the filter first.

Controllers/EmployeeController.cs (updated top section)

```
using WebApiDemo.Filters;

[Route("api/[controller]")]
[ApiController] [CustomAuthFilter] // <-- added
public class EmployeeController : ControllerBase
{
// ... same as before
}
```

Test in Postman / Swagger

Case 1 – No Authorization header sent:

[Postman – No Auth Header]

```
GET http://localhost:5001/api/employee
Headers: (none)
Response: 400 Bad Request
Body: Invalid request - No Auth token
```

Case 2 – Authorization header present but without 'Bearer':

[Postman – Invalid Token]

```
GET http://localhost:5001/api/employee
Headers: Authorization: abc123
Response: 400 Bad Request
Body: Invalid request - Token present but Bearer unavailable
```

Case 3 – Valid Bearer token:

[Postman – Valid Bearer Token]

```
GET http://localhost:5001/api/employee Headers:
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Response: 200 OK
Body: [ { Employee list JSON } ]
```

Task 3: Custom Exception Filter

Step 1: Install Required NuGet Package

Package Manager Console

```
Install-Package Microsoft.AspNetCore.Mvc.WebApiCompatShim
```

Step 2: Create CustomExceptionFilter

Create Filters/CustomExceptionFilter.cs. It implements `ExceptionHandler`. The `OnException` method captures the exception message, writes it to a log file on disk, and sets the Result to an `HttpResponseMessage` (via `WebApiCompatShim`) with status 500.

Filters/CustomExceptionFilter.cs

```

using System; using System.IO;
using Microsoft.AspNetCore.Mvc;
using
Microsoft.AspNetCore.Mvc.Filters;

namespace WebApiDemo.Filters { public class
CustomExceptionHandler : IExceptionHandler { public
void OnException(ExceptionContext context) { //
1. Capture exception details string message =
context.Exception.Message; string stackTrace =
context.Exception.StackTrace;

// 2. Write to a log file string
logPath = @"C:\Logs\error_log.txt";
Directory.CreateDirectory(@"C:\Logs");
File.AppendAllText(logPath,
$"[{DateTime.Now}] ERROR: {message}\n{stackTrace}\n\n");

// 3. Set the response to 500 Internal Server
Error context.Result = new ObjectResult(new {
error = "Internal Server Error", message =
message })
{ StatusCode
= 500
};

// Mark as handled so default error page is
skipped context.ExceptionHandled = true; }
}
}

```

Step 3: Register CustomExceptionHandler Globally in Startup.cs

Startup.cs – ConfigureServices

```

using WebApiDemo.Filters;
services.AddMvc(options =>
{ options.Filters.Add();
});

```

Step 4: Throw Exception in GET Action Method

Add ProducesResponseType(500) to the GET action and throw a test exception so we can verify the filter catches it.

Controllers/EmployeeController.cs – GET method updated

```
[AllowAnonymous]
[HttpGet]
[ProducesResponseType(typeof(List), 200)]
[ProducesResponseType(500)] // <-- added public
ActionResult> GetStandard() { // Intentional
exception to test CustomExceptionFilter throw new
Exception("Test exception from GET action");

// Normal code (unreachable during test)
// var list = GetStandardEmployeeList();
// return Ok(list);
}
```

Step 5: Test via Swagger and Verify Output

Open Swagger UI and click Execute on GET /api/employee. The filter intercepts the exception, logs it to file, and returns 500 with the error message.

[Swagger UI – Exception Response (500)]

```
GET /api/employee

Response: 500 Internal Server Error
{
  "error": "Internal Server Error", "message": "Test exception
from GET action"
}

-- Swagger UI shows response code 500 --
```

[C:\Logs\error_log.txt (log file on disk)]

```
File written to: C:\Logs\error_log.txt

[27-06-2026 10:35:42] ERROR: Test exception from GET action
at WebApiDemo.Controllers.EmployeeController.GetStandard()
in EmployeeController.cs: line 42
```